

E-Commerce API

Capstone 3

Version 7.2 Y

Capstone Description

This capstone is an e-commerce application, or an online store.

You will have a folder containing the API and the frontend website. You are only required to track your changes in the API. We recommend following your normal process of tracking changes, by tracking changes for the API. If you choose to make any frontend changes, create a separate repository for that app.

You should create a GitHub project board to manage your work. Use the project requirements below to create user stories on your board so that you can plan and manage your time throughout the project.

For this project you will assume the role of a backend developer for an existing website. The website is already operational and has been published as Version 1.

The website uses a Spring Boot API project for the backend server, with a MySQL database for data storage. You will be given the existing project code to modify. The starter code also includes the database script for the existing data structure.

You will get to choose your own frontend webstore for this application. The web app has 5 different stores to choose from:

- **The Easy Shop:** A general goods store with anything and everything you might need.
- **The Record Store:** A music store specializing in vinyl records and equipment.
- **The Grocery Store:** An online grocery portal to perform all your food needs.
- **The Clothing Store:** A hip website to order all your favorite brands.
- **The Video Game Store:** A video game store that sells direct to you.

You will be able to customize the stores name, add some custom products too! Be sure to use the correct database for the website you will be working on!

Since your code changes will all be in the API project you should rely heavily on Insomnia to test your application endpoints and your logic. However, a frontend website project is also available in your starter code, so that you can test your work, and see how your API is used on the web.

Bugs

The current API code is functional, but there are a few bugs in the current project. You will need to find and fix those bugs before adding any new features. You should use manual debugging and write unit tests to help fix these bugs.

New Features

Although you are not developing this project from scratch, you have been asked to develop several new features. You will need to plan, develop, and test the new features as you work on them.

General

Be sure to use diagramming tools and the GitHub project board to plan your work and to manage your time.

Demo

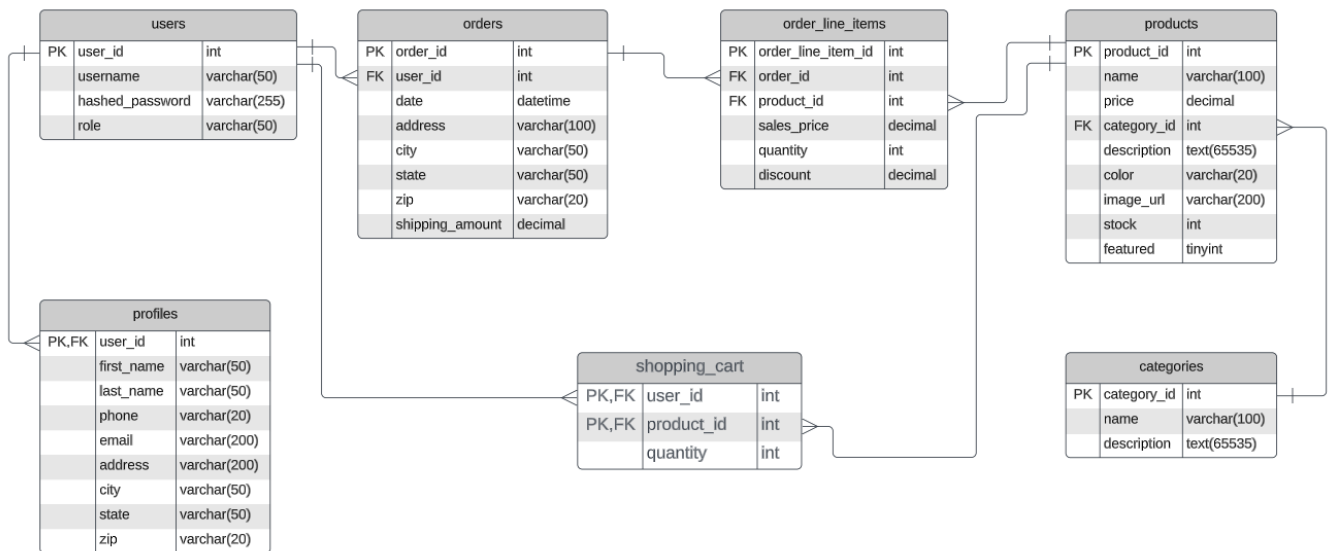
During the presentation of your final capstone, you will demonstrate the changes you have made throughout this project. You can do this by running the front-end web application and highlight the new features that you added. You should also be prepared to show some of the Insomnia scripts that you used to test your API.

Project Board & Starter Issues

When you first push your API project to GitHub, a set of starter issues is created for you automatically. One for each bug you need to find and fix. These become the first user stories on your project board, giving you a place to begin. As you plan the rest of your work, create your own issues (user stories) for each new feature, any bonus work you intend to build, and use the board to track your progress and manage your time throughout the project.

Capstone Setup

The database folder contains multiple database scripts. You will need to choose the .sql file based on the frontend store you choose. Open this script in MySQL Workbench and execute it to create the database as seen here.



The database script includes several products and 3 sample users (user, admin and george). The password for all demo users is `password`.

Set Up Your GitHub Project Board

Push your API project to a new GitHub repository. The first push automatically creates an issue for each capstone bug, so they're ready for you to start with.

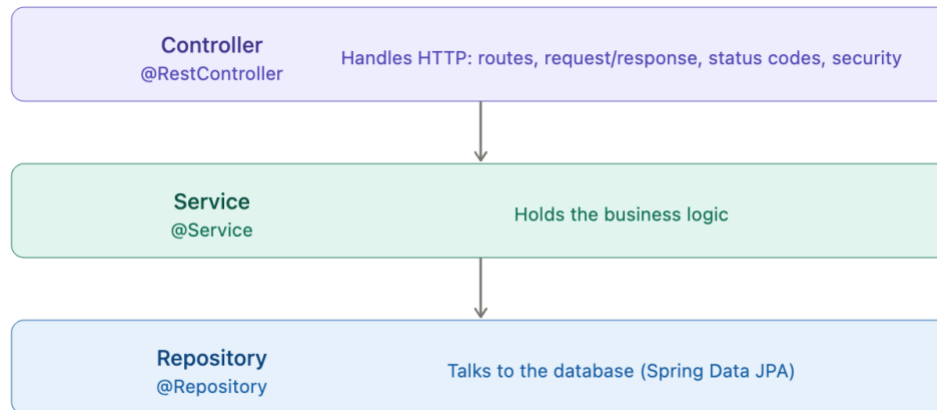
Create a GitHub Project (board) for your repository.

Add the starter issues to your board by either turning on the project's Auto-add to project workflow (filtered to label:capstone) so they appear automatically, or add them by hand.

Create your own issues / user stories for each feature (and any bonus work) you plan to build, and add them to the board.

Application structure

This project uses a layered Spring Boot architecture:



- The **models** (Category, Product, User, Profile, ShoppingCartItem, ...) are JPA entities mapped to the database tables for you.
- The **repositories** live in `org.yearup.repository` and extend `JpaRepository<Entity, IdType>`, so the basic data operations (`findAll()`, `findById()`, `save()`, `deleteById()`, and derived queries).
- The **services** live in `org.yearup.service` and hold the business logic. Some are already written; others are stubs for you to complete.
- The **controllers** live in `org.yearup.controllers` and depend on a service.

The application also uses an `AuthenticationController` to allow new users to register and existing users to login with the following urls:

Register

```
POST http://localhost:8080/register

{
  "username": "admin",
  "password": "password",
  "confirmPassword": "password",
  "role": "ADMIN",
}
```

Login

```
POST http://localhost:8080/login
```

```
{
  "username": "admin",
  "password": "password"
}
```

When a user logs in the API will return the relevant user information, including a JWT authentication token:

```
1 {
2   "token":
  "eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJhZG1pbSIzImF1dGgiOiJST0xFX0FETU0IiwiaXhwIjozcyMTYxOTMzZjQ.jTICEc13
  XPIpYC0uMXKaLYbCPmhmWc4hmVPnV_0Jf2xz0APMMtQ8c7jFPS1in9ZsLF4jUMyshxRNJeH0eeoAJQ",
3   "user": {
4     "id": 2,
5     "username": "admin",
6     "authorities": [
7       {
8         "name": "ROLE_ADMIN"
9       }
10    ]
11  }
12 }
```

The client application (insomnia and the sample website) will use this token to certify each request. You will need to add this token to insomnia requests for **all endpoints** which require a user to be logged in.

After running the database script, run the application and use Insomnia to verify that the existing functions work.

Application Requirements

The website allows users to browse products in various categories, add them to a shopping cart and check out to order the products. All of the features of the UI are fully functional. The changes and development will only need to be done in the backend Spring Boot Java API project.

The existing API code includes:

- * user registration and login
- * the ability to display products by category
- * search for or filter the products list.

Bugs

The search/filter functionality will have some bugs and may return incorrect search results. You should test different search and filter criteria to find and fix those bugs. (Include unit tests as part of this process)

Phase 1 - CategoriesController

The `CategoriesController` class has been created for you but none of the methods have been implemented yet. You must implement the code for each function and add the proper annotations in the controller. The controller should call the `CategoryService` where you will also implement the corresponding business logic.

Only administrators (users with the `ADMIN` role) should be allowed to insert, update or delete a category.

The `Category` model has been defined for you.

Requirements

The JSON for a category should be in the following format.

```
{
  "categoryId": 1,
  "name": "Electronics",
  "description": "Explore the latest gadgets and ..."
}
```

The following REST methods should be exposed by the API.

VERB	URL	BODY
GET	http://localhost:8080/categories	NO body
GET	http://localhost:8080/categories/1	NO body
GET	http://localhost:8080/categories/1/products	NO body
POST	http://localhost:8080/categories	Category
PUT	http://localhost:8080/categories/1	Category
DELETE	http://localhost:8080/categories/1	NO body

Phase 2 - Fix Bugs

The `ProductsController` endpoint is fully implemented with the following methods.

NOTE: only administrators are allowed to insert, update or delete products.

VERB	URL	BODY
GET	<code>http://localhost:8080/products</code>	NO body
GET	<code>http://localhost:8080/products/1</code>	NO body
POST	<code>http://localhost:8080/products</code>	Product
PUT	<code>http://localhost:8080/products/1</code>	Product
DELETE	<code>http://localhost:8080/products/1</code>	NO body

Bug 1

Users have reported that the product search functionality is returning incorrect results. Some products that should appear never show up in the list. You need to test the search logic to find and fix the bug(s). The search URL has the following query-string parameters

VERB	URL		
GET	<code>http://localhost:8080/products</code>		
	Key	Type	Filter
	<code>cat</code>	<code>int</code>	<code>categoryId</code>
	<code>minPrice</code>	<code>double</code>	<code>price (lower range)</code>
	<code>maxPrice</code>	<code>double</code>	<code>price (upper range)</code>
	<code>subCategory</code>	<code>String</code>	<code>subCategory</code>

Examples

```
http://localhost:8080/products?cat=1
http://localhost:8080/products?cat=1&subCategory=red
http://localhost:8080/products?minPrice=25
http://localhost:8080/products?minPrice=25&maxPrice=100
```

A good place to start is to call **GET** `/products` with no parameters at all and compare the results against the list of products in the database. Some products that exist are missing from the response.

Phase 2 - Fix Bugs

Bug 2

Some users have reported that when they edit a product, some of their changes don't stick.

For example, you can update a product's price or description and it saves correctly, but updating the product's stock appears to succeed (the request returns OK) while the value in the database never actually changes.

You need to find this bug and fix it so that users can fully update a product.

Optional Phase 3 - Shopping Cart

New Feature

Users should have the ability to add items to their shopping cart. This is a new feature that has not yet been implemented. The database already has a shopping cart table.

The shopping cart feature should only be available to logged in users. When an item is added to the shopping cart it gets added to the database for the current user. If a user logs out, the next time they log in the items that were added to the cart should still be in the cart.

The cart pieces are already created for you:

- A `CartItem` entity (one row per product in a user's cart, it stores `userId`, `productId`, and `quantity`), a `ShoppingCartRepository`, and the `ShoppingCart` / `ShoppingCartItem` models used for the response.
- A `ShoppingCartController` and a `ShoppingCartService` have also been created, but you need to implement the REST actions and the supporting business logic.

To build the cart for a user, load their `CartItem` rows and look up each `Product`, so the response includes the full product details.

VERB	URL	BODY
GET	<code>http://localhost:8080/cart</code>	NO body
POST	<code>http://localhost:8080/cart/products/15</code>	NO body
PUT	<code>http://localhost:8080/cart/products/15</code>	has body
DELETE	<code>http://localhost:8080/cart</code>	NO body

The **GET** action should return the shopping cart for the current user, including all shopping cart items (products). A method (`getCart`) has been started with sample code that shows how to get the logged-in user's username.

You must use the user lookup to get the `userId`, because you need it to look up that user's shopping cart items.

Optional Phase 3 - Shopping Cart

The JSON for a shopping cart should be in the following format:

```
{
  "items": {
    "1": {
      "product": {
        "productId": 1,
        "name": "Smartphone",
        "price": 499.99,
        "categoryId": 1,
        "description": "A powerful and feature-rich smartphone
for all your communication needs.",
        "subCategory": "Black",
        "stock": 50,
        "imageUrl": "smartphone.jpg",
        "featured": false
      },
      "quantity": 2,
      "discountPercent": 0,
      "lineTotal": 999.98
    },
    "15": {
      "product": {
        "productId": 15,
        "name": "External Hard Drive",
        "price": 129.99,
        "categoryId": 1,
        "description": "Expand your storage capacity and backup y
our important files with this external hard dr
ive.",
        "subCategory": "Gray",
        "stock": 25,
        "imageUrl": "external-hard-drive.jpg",
        "featured": true
      },
      "quantity": 1,
      "discountPercent": 0,
      "lineTotal": 129.99
    }
  },
  "total": 1129.97
}
```

Optional Phase 3 - Shopping Cart

The **POST** action is used to add a new product to the shopping cart. Note that the URL is `http://localhost:8080/cart/products/15` the number is the id of the product that you want to add to the cart. If the current user's shopping cart does not yet include the product id, you should add (insert) the `userId` and `productId` to the shopping cart table with a quantity of 1.

If the user already has the specified product in their cart, then you should update the shopping cart table, and update the quantity to increase it by 1.

The **DELETE** method should clear the shopping cart. Delete all of the current user's items from the shopping cart table.

Optional / Bonus - PUT

The **PUT** method is similar to the POST method, but an id should be included with the Request. The URL is `http://localhost:8080/cart/products/15`. The `productId` is the id that should be updated in the database. The request body will include the new quantity that should be updated in the database.

Body format:

```
{
  "quantity": 3
}
```

The shopping cart item should only be updated if the user has already added the product to their cart.

Optional Phase 4 - User Profile

When a user registers for an account, a user profile record is also created (in the `profiles` table). A user should be able to view and update their profile. You need to create a `ProfileController` and add the following methods.

VERB	URL	BODY
GET	<code>http://localhost:8080/profile</code>	NO body
PUT	<code>http://localhost:8080/profile</code>	Profile body

A `ProfileService` and `ProfileRepository` already exist and `ProfileService.create(...)` is used to create the profile when a user registers.

You will need to add the methods you need (for example, to get a profile by user id and to update it) to the `ProfileService`, then call them from your new controller.

Take time to plan and design the requirements of this feature. Then write the implementation code.

Optional Phase 5 - Checkout

New Feature

When a user is ready to check out you need to convert their shopping cart into an order. This feature will require a significant amount of planning, because none of the code required for this feature exists in the starter code.

Following the same layered pattern as the rest of the project, you will create the model(s), a repository, a service, and a controller for orders. An order has an `order` record plus one order-line-item record per product. The `orders` and `order_line_items` tables already exist in the database script.

VERB	URL	BODY
POST	<code>http://localhost:8080/orders</code>	NO body

The **POST** method does not require a body. It should retrieve the current user's shopping cart, create and insert a new order into the `orders` table, then create an order line item for each shopping cart item and save each one.

Once the order has been created you should clear the shopping cart.

Future Versions

Take some time with your instructor or other class members to discuss what other features might be worth developing in future versions. Consider other e-commerce websites that you have used (such as Amazon) and think about features that are available.

There is no need to fully plan or implement these features but make a list of these features and rank them in order of priority.

What changes do you think would need to be made to your project to implement these features?

Other General Project Requirements

Your project must also meet the following requirements:

Git Repository

- Your code must be in a public GitHub repository
- The repository must contain an appropriate Git commit history
 - * *Minimally*, you should have a commit for each meaningful piece of work completed
- It must contain an informative README file that:
 - * Describes your project
 - * Includes images of your application screens
 - * Describes/shows one interesting piece of code from your project

Class Demonstrations

Each student will be given 10 minutes to demonstrate their project to the class on "project demonstration day". During this time, you will:

- Present your application - run through the website ordering process
 - * You should also demonstrate how you can use Insomnia to interact with and test your API
- Describe / show one interesting piece of code that you wrote
- Answer questions from the audience if time permits